Initiation au Développement : Algorithmique Fondamentale, Variables et Structures de Contrôle

Thème principal : Fondements de l'algorithmique : gestion de la mémoire RAM via les variables, instructions d'affectation, structures conditionnelles (Si/Sinon) et itératives (Pour/TantQue)

Synthèse exécutive : Ce cours pose les fondations absolues du développement logiciel et de la pensée algorithmique pour les étudiants en BUT Informatique. Il résout le problème fondamental de la traduction d'un processus décisionnel concret ou d'un calcul complexe en une séquence d'instructions formelles, explicites et déterministes exécutables par un processeur. Le cours détaille la modélisation de la mémoire vive (RAM) à travers le concept de variables (caractérisées par leur quadruplet Nom, Type, Valeur, Adresse), la mécanique d'affectation destructive, l'aiguillage du flux d'exécution au moyen de prédicats booléens et de structures conditionnelles (Si / Alors / Sinon), ainsi que l'automatisation des traitements répétitifs grâce aux structures itératives bornées (Pour) et non bornées (TantQue).

1. Mémorisation et Manipulation des Données : Les Variables en RAM

En informatique, le concept de variable se distingue radicalement de sa définition mathématique. Il ne s'agit pas d'une inconnue abstraite dans une équation statique, mais d'une case mémoire concrète localisée dans la mémoire vive (RAM). Une variable est obligatoirement définie par un quadruplet : son Nom (qui permet au développeur d'y faire référence), son Type (qui indique au système combien d'octets réserver et comment encoder la donnée), sa Valeur (le contenu binaire stocké à un instant t) et son Adresse physique (l'emplacement hexadécimal exact en RAM). Lors de l'exécution d'une instruction d'affectation (par exemple x ← 10 ou x ← x + 1), l'Unité Centrale de Traitement (CPU) procède en deux étapes strictement chronologiques : premierement, elle évalue la valeur globale de l'expression située du côté droit de la flèche en lisant les valeurs actuelles en RAM ; deuxièmement, elle vient inscrire ce résultat final dans la case mémoire identifiée à gauche. Ce processus est destructif : la nouvelle valeur écrase définitivement l'ancienne. Un piège classique en examen concerne l'utilisation d'une variable non initialisée. Si un algorithme tente d'évaluer une expression contenant une variable qui n'a reçu aucune affectation préalable, le programme va lire des bits résiduels restés en RAM (valeur 'poubelle'). Cela provoque des erreurs d'exécution ou des résultats totalement erronés.

- Variable : Emplacement mémoire étiqueté réservé dans la mémoire vive (RAM) pour stocker une valeur modifiable au cours de l'exécution d'un programme.
- Quadruplet d'une variable : Élément indissociable composé de son Nom (identificateur usuel), son Type (format d'encodage et taille réservée en octets), sa Valeur actuelle et son Adresse mémoire physique.
- Typage formel : Spécification explicite du domaine de valeurs autorisées (Entier, Réel, Chaîne de caractères, Booléen) qui détermine la quantité d'octets réservée et les opérateurs valides.
- Affectation (← ou =) : Opération fondamentale d'écriture en mémoire qui évalue l'expression située à droite du symbole et remplace de manière irrémédiable la valeur stockée dans la variable située à gauche.

Cas concret enseignant : *La métaphore de l'armoire d'archivage à tiroirs : Imaginez la mémoire RAM comme une grande armoire remplie de tiroirs. Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à ajuster sa profondeur selon son type ('Entier'). L'instruction d'affectation 'solde_compte ← 150' consiste à ouvrir ce tiroir spécifique, à jeter le bout de papier qui s'y trouvait déjà (l'ancienne valeur), et à y déposer la feuille portant le nombre 150.*

2. Prise de Décision : Les Structures Conditionnelles (Si / Sinon)

Par défaut, un processeur exécute les instructions d'un programme de façon purement séquentielle, ligne après ligne. Pour introduire du comportement dynamique et adaptatif, on fait appel aux structures conditionnelles. Lors du franchissement d'un bloc 'Si (Condition) Alors [Bloc A] Sinon [Bloc B]', la machine commence par évaluer la condition booléenne. Si la condition s'évalue à VRAI, le processeur exécute exclusivement les instructions

contenues dans le [Bloc A], puis reprend l'exécution après le bloc conditionnel, en ignorant totalement le [Bloc B]. À l'inverse, si la condition s'évalue à FAUX, le [Bloc A] est sauté et seul le [Bloc B] (s'il est présent) est exécuté. Pour composer des conditions multiples, il est indispensable de maîtriser la logique de Boole et les tables de vérité : l'opérateur 'ET' (AND) requiert que l'ensemble des sous-conditions soient simultanément vraies pour que la condition globale soit VRAIE. L'opérateur 'OU' (OR) exige seulement qu'au moins une des sous-conditions soit vraie. L'opérateur 'NON' (NOT) inverse la valeur de vérité d'une expression.

- Structure d'Alternative (Si / Alors / Sinon) : Aiguillage logique qui détourne le flux d'exécution séquentiel d'un programme en fonction du résultat d'un test logique.
- Expression Booléenne (Prédicat) : Expression évaluée par le processeur dont le résultat est strictement binaire : VRAI (True) ou FAUX (False).
- Opérateurs Logiques (ET, OU, NON) : Connecteurs permettant d'assembler des conditions complexes régies par leurs tables de vérité respectives.

Cas concret enseignant : *Le cas réel du Distributeur Automatique de Billets (DAB) : Avant d'éjecter l'argent liquide, la machine évalue la condition composée : SI (solde_client >= montant_demandé) ET (billets_disponibles_dans_automate >= montant_demandé). Si cette expression globale est évaluée à VRAI, les billets sont délivrés et le compte est débité ; SINON, l'opération est refusée, un message d'erreur est affiché sur l'écran et la carte bancaire est restituée à l'usager.*

3. Répétition et Automatisation : Les Structures Itératives (Boucles)

La puissance de calcul de l'informatique réside dans sa capacité à traiter des opérations répétitives sans défaillance à très haute vitesse. On distingue deux structures itératives fondamentales : 1. La boucle 'Pour' (Structure bornée) : Elle est privilégiée lorsque le nombre d'itérations est connu d'avance. Elle intègre de façon native l'initialisation d'un compteur, le test de limite et l'incrémentation automatique à chaque passage (ex: parcourir les N éléments d'un tableau). 2. La boucle 'TantQue' (Structure non bornée) : Elle est employée lorsque le nombre de répétitions dépend d'un événement dynamique (ex: attendre qu'un utilisateur saisisse un mot de passe valide). La condition est évaluée AVANT chaque passage. Si elle est vraie, le corps de la boucle est exécuté ; si elle est fausse dès le départ, le corps de la boucle n'est jamais exécuté. La règle d'or d'une boucle TantQue est l'obligation de modifier, à l'intérieur du corps de la boucle, au moins une variable intervenant dans la condition d'arrêt. Si cette variable reste inchangée, la condition demeure éternellement VRAIE, provoquant ce qu'on appelle une 'boucle infinie' qui bloque l'exécution du programme et consomme 100% des ressources CPU.

- Boucle Pour (FOR) : Structure itérative bornée, utilisée lorsque le nombre exact de répétitions est connu de façon déterministe avant le début de la boucle.
- Boucle TantQue (WHILE) : Structure itérative non bornée, répétée de manière dynamique tant qu'une condition logique demeure VRAIE.
- Variable de Contrôle & Incrémentation : Modification explicite d'une variable au sein du corps d'une boucle pour garantir l'atteinte de la condition d'arrêt.

Cas concret enseignant : *La métaphore des consignes de l'entraîneur d'athlétisme : Si l'entraîneur ordonne au coureur 'Fais 10 tours de piste', il s'agit d'une boucle POUR (le nombre 10 est fixé au départ et la sortie est garantie après 10 itérations). S'il ordonne 'Cours tant qu'il ne pleut pas', il s'agit d'une boucle TANT QUE (la fin de l'exercice dépend d'une condition météo évaluée à chaque tour).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi l'instruction 'x = x + 1' est-elle valide en programmation alors qu'elle est mathématiquement impossible ?
Réponse du professeur : En programmation, le symbole '=' (ou '←') n'est pas une égalité mathématique statique, mais un opérateur d'affectation dynamique. Le processeur résout d'abord le membre de droite : il lit la valeur actuelle stockée dans la variable 'x' en RAM, lui ajoute 1, puis vient réécrire ce nouveau résultat dans la case 'x'. L'ancienne valeur est écrasée. C'est l'opération classique d'incrémentation.

Question étudiant : Que se passe-t-il concrètement si j'oublie d'actualiser la variable de contrôle dans le corps d'une boucle 'TantQue' ?
Réponse du professeur : Si la variable évaluée dans le test de la boucle n'est jamais modifiée à l'intérieur du bloc d'instructions, la condition reste éternellement VRAIE. Le programme entre alors dans une 'boucle infinie' : il réexécute le même bloc d'instructions indéfiniment, ne rend plus la main et plante ou gèle le système.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme accumulée de tous les entiers compris entre 1 et N inclus (Somme = 1 + 2 + ... + N).

Résolution pas à pas :
Étape 1 : Déclaration des variables nécessaires avec typage explicite : - N : Entier (valeur limite saisie par l'utilisateur) - i : Entier (compteur / variable de contrôle de boucle) - somme : Entier (accumulateur) Étape 2 : Interaction utilisateur et saisie : Afficher "Veuillez saisir un nombre entier positif N : " Lire N Étape 3 : Initialisation stricte de l'accumulateur : somme ← 0 // Indispensable pour éliminer toute valeur 'poubelle' en RAM Étape 4 : Structure itérative de calcul (Boucle Pour) : Pour i allant de 1 à N par pas de 1 Faire somme ← somme + i FinPour Étape 5 : Affichage du résultat final : Afficher "La somme des entiers de 1 à ", N, " est égale à : ", somme

Résultat : Pour une valeur saisie N = 5, le déroulement pas-à-pas en mémoire RAM est le suivant : - Initialisation : somme = 0 - Itération 1 (i=1) : somme ← 0 + 1 = 1 - Itération 2 (i=2) : somme ← 1 + 2 = 3 - Itération 3 (i=3) : somme ← 3 + 3 = 6 - Itération 4 (i=4) : somme ← 6 + 4 = 10 - Itération 5 (i=5) : somme ← 10 + 5 = 15 Le programme affiche : 'La somme des entiers de 1 à 5 est égale à : 15'. L'algorithme est déterministe et valide.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale de choix entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR est utilisée lorsque le nombre d'itérations est connu à l'avance (structure bornée avec compteur automatique). La boucle TANT QUE est utilisée lorsque le nombre d'itérations est inconnu et dépend d'une condition dynamique évaluée à chaque passage (structure non bornée).

Q : Que vaut la variable A après l'exécution exacte de la séquence : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?

R : A vaut 8. Déroulement mémoire : A prend 5, B prend 3. Puis A ← 5 + 3 = 8. Enfin B ← 8 - 3 = 5. À la fin de la séquence, A contient la valeur 8 (et B contient 5, ce qui a réalisé une permutation).

Q : Pourquoi l'absence d'initialisation d'une variable d'accumulation (ex: somme ← 0) fait-elle perdre l'intégralité des points en examen ?

R : Car sans initialisation, la variable contient la valeur résiduelle ('poubelle') présente à cette adresse mémoire en RAM avant l'exécution. Faire 'somme ← somme + i' ajouterait 'i' à une valeur indéterminée, rendant le résultat du calcul complètement faux et imprévisible.